How the System Finds and Edits the Right File

Oct 18, 2026 · @Toqeer

Use PostgreSQL with the pgvector extension, not a separate vector database, and store one embedding per file instead of code chunks. The Locator searches a file manifest, loads only the 1 to 3 affected files, and the sandbox still tests the full project.

How the system finds the file

The Locator does not search code. It searches a file manifest, one row per file in Postgres:

Column	Example
path	`components/ProductCard.tsx`
role	component, content, page, style, config or api
summary	'Product card with image, price and Buy Now button' (written by a small LLM)
exports / imports	`ProductCard` / `data/site.json`, `Button.tsx`
content hash	used to detect changes
embedding	vector of path + summary + exports (not the code)

For a request like 'change the Buy Now button to Add to Cart and make it gold', retrieval works in three steps:

1. **Shortlist candidates.** Combine three signals: vector search on the summaries, keyword and path matching ('button', 'cart'), and the import graph (what imports or is imported by the top hits).
2. **Let the Locator decide.** A small LLM sees only the shortlist (path, summary, exports, role) and returns 1 to 5 files.
3. **Use conventions to make it predictable.** The Planner enforces one rule: all text and labels live in `data/site.json`, and components only render it. Label changes then go to the data file, while layout or style changes go to the component. The `role` column makes this routing easy.

For this example the result is `data/site.json` (the label) plus `components/ProductCard.tsx` or the Tailwind config (the gold styling).

Vector database or PostgreSQL?

Use PostgreSQL with pgvector.

- **File-level embeddings, not chunks.** Chunks lose the meaning of the whole file, and raw JSX matches natural language badly. Embedding path, summary and exports works far better, and it keeps one row per file with no chunk-to-file mapping.
- **One database.** A version row, its file rows and their embeddings can be written in one transaction. A separate vector database (Pinecone, Qdrant) adds a second service and sync problems for no benefit at this scale.
- **Vectors can wait.** A generated site has about 20 to 60 files, and a manifest costs about 50 tokens per file, so the whole manifest is roughly 1 to 3k tokens. For the MVP the Locator can read the entire manifest directly. Add pgvector when projects get larger, behind the same interface.

The edit flow, step by step

1. The Locator returns the affected files, as above.
2. Only those files are loaded from GitHub (through the Committer and GitHub MCP, or a cached clone), not from the database. Git holds the contents; Postgres holds only the manifest and metadata.
3. The Editor produces diffs for those 1 to 3 files, and can request one more file if it needs it.
4. The sandbox tests the full project, not just the changed file. A change in one file can break another (for example a renamed export), so the sandbox gets the whole project from the repo with the diffs applied and runs `tsc`, lint, build and Playwright. The LLM still sees only the changed files. Cached dependencies keep this fast, and Playwright can be limited to the affected pages.
5. If the tests fail, the Fixer gets only the error log and the failing files (max 3 tries). If it still fails, the attempt stays on a draft branch and main is untouched.
6. If the tests pass, the Committer commits only the changed files in one commit to main and tags it vN.
7. The database is updated:

- Insert a new `versions` row with the commit SHA, the linked requirements version and the message that triggered it.
- Copy the unchanged file rows to the new version.
- For changed files only, compare hashes, then regenerate the summary, exports, imports and embedding.

Backend changes

The mechanism is identical. A request like 'add validation to the contact form API' retrieves the API route file plus its related type or schema file through the import graph. The two MVP profiles (static-html and vite-react-tailwind) have no real backend, so this applies once Next.js profiles are added.